

The Autoresearcher Intern: Evidence-Gated Human–AI Collaboration for Scientific Workflow Automation

Anonymous authors

Paper under double-blind review

Abstract

We present the *autoresearcher intern*, a repository-centered pipeline for human-guided scientific prototyping with language-model agents. Rather than treating current agents as fully autonomous scientists, the *autoresearcher intern* decomposes research into bounded planning, execution, review, summarization, and checkpoint-supervision roles. It stores plans, commands, metrics, artifacts, reviews, and claim boundaries as auditable repository state, and escalates high-stakes decisions to a stronger model such as GPT-5.5-Pro. Across two live reinforcement-learning case studies, the system produced scoped positive evidence, decision-relevant negative results, and stopping decisions that prevented unsupported scaling. We argue that this research-intern paradigm is a practical human-AI collaboration model for fast, large-scale prototyping of scientific ideas.

1 Introduction

Scientific work is often described through breakthroughs: a new idea, a new theorem, a surprising experimental result, or a new algorithm. In daily computational research, however, progress often depends on a more mundane and expensive loop: formulate a hypothesis, translate it into code, build a minimal diagnostic, choose a fair baseline, run the experiment, inspect artifacts, update the hypothesis, and write down what happened. For machine-learning researchers, much of this loop is not blocked by creativity alone. It is blocked by engineering bandwidth. A scientist may have many plausible ideas, but each idea requires environment setup, small benchmark construction, scripts, metrics, baselines, validation commands, plots, result interpretation, and careful documentation. Many ideas should be stopped early, but weak evidence is easy to miss when it lives in scattered logs and unstructured model summaries.

This bottleneck motivates language-model systems for scientific discovery. Recent systems branded as “AI scientists,” “autonomous researchers,” or “autoresearchers” attempt to automate increasingly large portions of the research process, from idea generation to coding, experimentation, and manuscript writing (Lu et al., 2024; Yamada et al., 2025; Schmidgall et al., 2025; Ghareeb et al., 2025; Ghafarollahi & Buehler, 2024). These systems demonstrate that language models can coordinate tools, modify repositories, execute experiments, and produce research-like artifacts. At the same time, we believe the phrase “autoresearcher” is currently aspirational. Today’s systems are not yet reliable end-to-end scientists. Their outputs still require human framing, expert review, artifact checking, baseline skepticism, and claim control. Benchmarks of machine-learning and software-engineering agents report substantial variability, long-horizon planning failures, hallucination risks, and dependence on environment design (Huang et al., 2023; Jimenez et al., 2023; Yang et al., 2024). Agent Laboratory explicitly studies human feedback during stages of an automated research pipeline, underscoring that human guidance remains valuable even when agents can run substantial workflows (Schmidgall et al., 2025).

This paper presents a prototype of that paradigm: the *autoresearcher intern*. The system is a repository-centered orchestration pipeline for human–AI scientific prototyping. A project begins with a human-written charter specifying the research goal, main hypothesis, scope, metrics, success criteria, failure criteria, and compute budget. A local supervisor agent proposes one next experiment or chooses to pivot, stop, request human or request a stronger

model review. An executor agent implements and runs the experiment under a wall-clock budget. A reviewer agent audits the result and states whether automatic continuation is allowed. A summarizer agent writes progress reports and claim boundaries. A deterministic orchestrator validates every structured artifact, commits project state, enforces stop conditions, and escalates high-stakes decisions to stronger model such as GPT-5.5-Pro.

The central design principle is *evidence-gated co-autonomy*. The *autoresearcher intern* allows agents to propose and execute experiments, but not to silently convert activity into scientific progress. Claims must be grounded in machine-readable result files, declared artifacts, baseline metrics, reviewer verdicts, and metric ledgers. Continuation is allowed only when both deterministic validators and model-based reviewers accept the evidence. Pivots, stop decisions, ambiguous reviews, timeouts, repeated weak passes, protected-file drift, and publication-oriented claims can trigger a stronger model or human review. The repository, rather than any chat transcript, is the authoritative memory.

We evaluate the *autoresearcher intern* through two live reinforcement-learning research projects. The first, which we call **Soft Reward-to-Goal Conversion**, tests whether soft successor-measure terminal marginalization can preserve reward-to-goal equivalence while reducing the variance of sampled terminal success events. In this project, the *autoresearcher intern* progressed from one-state variance diagnostics to tabular equivalence checks, River-Swim coverage tests, FourRooms vector-goal checks, and a low-rank auxiliary-goal negative result. It produced a scoped internal write-up. The second, which we call **Stochastic Long-Horizon Goal Reaching**, tests stochastic extensions of Transitive RL (TRL). This project produced useful diagnostics, but ultimately stopped after showing that apparent stochastic-TRL improvements were explained by fairer baselines or representation/context sufficiency rather than a distinct algorithmic advantage. We use Soft Reward-to-Goal Conversion as the main case study and summarize Stochastic Long-Horizon Goal Reaching in the appendix.

This paper makes four contributions:

- We introduce evidence-gated co-autonomy, a human–AI collaboration pattern in which agents can run bounded research loops but continuation depends on structured evidence and explicit gates.
- We describe a concrete architecture for repository-centered scientific memory, including structured plans, results, reviews, metric ledgers, progress summaries, and checkpoint packets.
- We show how a stronger supervisor model can be used selectively as a senior checkpoint reviewer rather than as the default executor.
- We report two live case studies showing that the system can produce both positive and negative scientific evidence, including stopping decisions that prevent unsupported scaling.

2 Related Work

2.1 Autonomous AI scientists and scientific agents

The most direct comparison is with systems that attempt broad automation of scientific discovery. The AI Scientist proposes an end-to-end pipeline for idea generation, experiment execution, manuscript writing, and automated review (Lu et al., 2024). AI Scientist-v2 extends this direction with agentic tree search and workshop-level paper generation (Yamada et al., 2025). Agent Laboratory studies LLM agents as research assistants across literature review, experimentation, and report writing, while allowing users to provide feedback (Schmidgall et al., 2025). Robin explores a multi-agent system for automating scientific discovery in biology (Ghareeb et al., 2025); SciAgents uses multi-agent graph reasoning for scientific discovery (Ghafarollahi & Buehler, 2024). Surveys have begun to organize this area as LLM-based scientific agents or as movement from automation toward autonomy in scientific workflows (Ren et al., 2025; Zheng et al., 2025; Zhang et al., 2025).

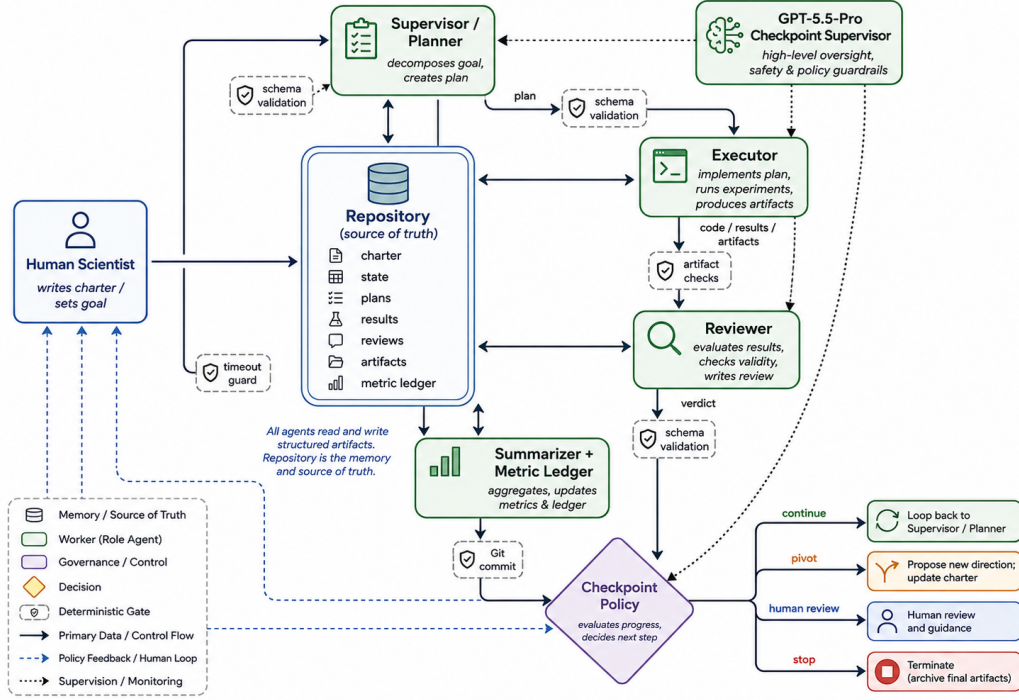


Figure 1: Architecture of the *autoresearcher intern*. The repository is the source of truth; agents are workers that read and write structured artifacts under deterministic validation.

2.2 Language-agent scaffolding, tool use, and reflection

The *autoresearcher intern* builds on general language-agent scaffolding. ReAct interleaves reasoning and acting so that language models can use tools and update plans from observations (Yao et al., 2022). Toolformer studies models that decide when and how to call external tools (Schick et al., 2023). Tree of Thoughts explores search over intermediate reasoning states (Yao et al., 2023). Reflexion shows how agents can improve by storing verbal reflections over prior trials (Shinn et al., 2023). Voyager combines an automatic curriculum, executable skill library, and iterative prompting for open-ended embodied learning (Wang et al., 2023).

2.3 Coding agents, benchmarks, and reproducible experimentation

Scientific prototyping often looks like software engineering: the agent must navigate a repository, edit code, run commands, and interpret failures. SWE-bench evaluates language models on real GitHub issues (Jimenez et al., 2023). SWE-agent argues that agent-computer interface design is central to automated software engineering performance (Yang et al., 2024). MLAGentBench evaluates language agents on machine-learning experimentation and identifies challenges such as long-term planning and hallucination (Huang et al., 2023). Work on SWE-Bench+ further highlights that apparent benchmark success can be affected by leakage and weak tests (Aleithan et al., 2024). These findings motivate the *autoresearcher intern*'s deterministic validators, artifact checks, and protected-file drift gates.

3 Method: The Autoresearcher Intern Pipeline

3.1 Design goals

The *autoresearcher intern* is designed for early-stage scientific prototyping. It is not a benchmark optimizer, an autonomous paper factory, or a replacement for expert review. Its goals are: (1) translate rough ideas into falsifiable small experiments; (2) maintain an auditable record of plans, commands, metrics, and evidence; (3) run quickly enough that many ideas can be screened; (4) stop weak directions before they consume large compute; and (5) help the human scientist preserve claim boundaries. These goals lead to four design choices.

First, the system is *bounded*. Experiments are intentionally small. The default executor timeout is 30 minutes, and the charter can prohibit large training runs, large downloads, expensive API calls, or benchmark-scale experiments without approval. This makes the pipeline useful for fast and large-scale prototyping of ideas: a scientist can test many hypotheses at low cost and reserve expensive validation for ideas that survive early falsification.

Second, the system is *role-specialized*. Planning, execution, review, summarization, and senior checkpointing are separated. This prevents a single agent from both producing and uncritically accepting its own result. It also lets different roles operate under different permissions: the executor can modify experiment code, while the reviewer and supervisor normally run read-only.

Third, the system is *repository-centered*. Scientific memory lives in versioned files, not in a chat transcript. Every role is instructed that repository files are the source of truth. This makes it possible to audit the project after many iterations, reset or replace an agent, and still recover the research state.

Fourth, the system is *evidence-gated*. Every transition is mediated by validators or reviews. A result that omits artifact paths, fails schema validation, or lacks a reviewer verdict cannot silently advance the project. A stop or pivot decision can trigger stronger model checkpoint supervision. A draft intended for external use must include red-line unsupported claims and a pre-publication review checklist.

3.2 Human charter as the root of research intent

Every project begins with a human-written charter. The charter is intentionally short but semantically important. It specifies the research goal, main hypothesis, initial scope, primary metrics, success criteria, failure criteria, runtime budget, and first-experiment guidance. This document is the contract between the human scientist and the agent system.

The charter serves several functions. It prevents the local supervisor from drifting toward unrelated experiments. It defines what early evidence should count. It states what would disprove or block the idea. It encodes compute limits and red lines. It also makes human taste explicit: the scientist can say which baselines matter, which settings are too expensive, and which results would be scientifically meaningful. In our case studies, charters prevented premature moves to large RL benchmarks and forced the pipeline to begin with exact-DP tabular diagnostics.

3.3 Role-specialized agents

The *autoresearcher intern* uses five roles. The *supervisor* reads the charter, state, metric ledger, latest results, and latest reviews. It decides whether to continue, pivot, stop, request human input, or request a stronger model for review. If it continues, it proposes one small experiment with success criteria, failure criteria, required outputs, and estimated runtime. The motivation is to make planning falsifiable. A vague instruction such as “try more variants” is not acceptable; the next experiment must test a named claim.

The *executor* receives the plan and runs the experiment. It may edit code and create artifacts, but it must write a structured result file and a human-readable summary. It records exact

commands, metrics, baseline metrics, artifacts, interpretation, known failures, and next questions. The executor is not allowed to change the research goal, delete prior results, or run large experiments outside the budget. If it cannot complete the plan, it writes a failed, blocked, or timeout result rather than leaving the project ambiguous.

The *reviewer* audits the result after execution. It checks whether the required files exist, whether the artifact paths are valid, whether the baseline is fair, whether success criteria were satisfied, whether failure criteria were triggered, and whether the interpretation overclaims the evidence. It returns a verdict and a boolean indicating whether automatic continuation is allowed. The reviewer exists because executor summaries are claims, not facts.

The *summarizer* periodically writes progress reports. These reports are not only for convenience. They compress the research state into a human-readable lab notebook: current status, experiment ledger, main findings, limitations, recommended human decision, and files to inspect. Summaries also become inputs to checkpoint packets.

Finally, the *checkpoint supervisor* is a stronger model, e.g., GPT-5.5-Pro, invoked only at important gates. It is not used for every routine action (because it is expensive). Instead, it reviews compact packets when the local loop proposes a pivot or stop, accumulates weak passes, reaches a cadence checkpoint, times out, encounters protected-file drift, or approaches a publishable claim. This senior-review pattern uses strong reasoning where it is most valuable without making the entire loop expensive or brittle.

3.4 Repository-centered memory and data formats

The repository is organized as a lab notebook with machine-readable structure. Each project contains a charter, state file, plans, results, reviews, decisions, progress summaries, metric ledgers, pro-review packets, and artifacts. The state file tracks iteration count, status, failure streak, no-progress rounds, last decision, last checkpoint, protected-file drift, and whether human review is required. Plans are stored as both JSON and Markdown. Results are stored as JSON plus human summaries. Reviews are stored as JSON plus Markdown. Metric ledgers summarize decision-relevant quantities across iterations.

The result schema is central. A result must name the experiment id, status, claim tested, commands run, metrics, baseline metrics, artifact paths, interpretation, known failures, and next questions. This schema forces the executor to connect actions to evidence. The review schema forces the reviewer to state a verdict, evidence checked, reasons, required fixes, and risk flags.

3.5 Grounding claims in evidence

Scientific overclaiming is a major failure mode for agentic workflows. The *autoresearcher intern* addresses this by requiring every claim to be linked to result artifacts. Each experiment begins with success and failure criteria. Each result records the claim tested and baseline metrics. The reviewer checks whether the interpretation follows from the metrics. The metric ledger extracts important scalars, positive signals, and negative signals across iterations. This structure creates a claim-to-evidence map. The system does not treat a successful run as scientific success. It asks: what claim did the experiment test, against what baseline, with what artifact-backed evidence?

3.6 Reflection, self-checking, and checkpoint escalation

The *autoresearcher intern* uses reflection in a constrained way. Agents do not merely write free-form thoughts. The reviewer reflects on the executor’s claims by checking artifacts and baselines. The summarizer reflects on the project trajectory by compressing evidence into status and limitations. The supervisor reflects on whether continuation is justified. The stronger model reflects on local decisions at higher-stakes checkpoints.

The system also distinguishes between recoverable failures and scientific failures. Invalid JSON, missing artifacts, timeout, and bridge failures are operational failures; they are recorded and may be retried. Negative metrics, baseline equivalence, and triggered failure

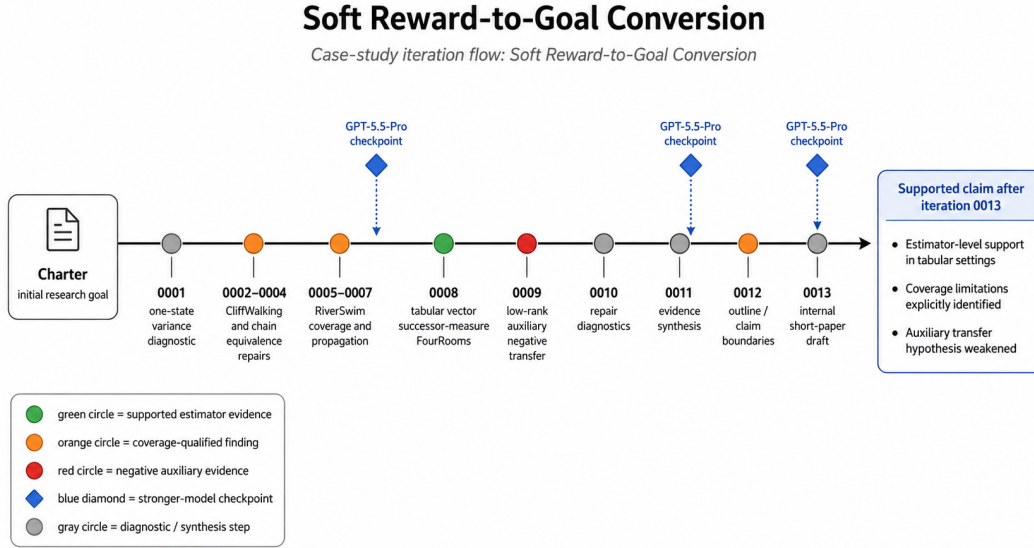


Figure 2: Case-study iteration flow for Soft Reward-to-Goal Conversion. The important story is not a single benchmark number but a sequence of evidence-gated decisions that narrow the supported claim.

criteria are scientific evidence; they may justify stopping. This distinction prevents the loop from confusing infrastructure problems with research conclusions.

4 Evaluation Protocol

We evaluate the *autoresearcher intern* through live use on two research projects proposed by the human scientist. We ask whether the pipeline exhibits behaviors required of a useful research intern:

1. **Translation.** Can the system convert a rough idea and charter into concrete experiments?
2. **Execution.** Can it implement and run small diagnostics with required artifacts?
3. **Grounding.** Does it tie claims to metrics, commands, baseline comparisons, and evidence files?
4. **Epistemic discipline.** Does it preserve negative evidence and avoid unsupported scaling?
5. **Autonomy control.** Does it know when to continue locally and when to escalate, stop, or ask for human judgment?

The two projects are both reinforcement-learning-style computational ideas, but they differ in outcome. Soft Reward-to-Goal Conversion produced scoped positive estimator evidence and scoped negative auxiliary evidence, culminating in an internal short-paper draft. Stochastic Long-Horizon Goal Reaching produced useful diagnostics but ultimately stopped after stronger baselines explained apparent algorithmic gains. This contrast is useful: an autoresearcher intern should not be judged only by whether it finds positive results. It should also be judged by whether it can narrow claims and terminate weak directions.

5 Main Case Study: Soft Reward-to-Goal Conversion

5.1 Motivation and starting idea

The first project began from a simple but practically important question in goal-conditioned reinforcement learning. Suppose a researcher wants to transform an ordinary reward-

maximization problem into a goal-reaching problem by adding an artificial “success” goal. A sampled conversion can create sparse terminal success events: the learner sees rare samples indicating that the artificial goal was reached. This preserves the idea in expectation, but it can introduce high-variance targets, especially when rewards are small or the discount factor is close to one.

The human scientist proposed a soft alternative. Instead of sampling terminal success events, train the artificial success-goal head with expected terminal mass:

$$y(s, a, g_+) = (1 - \gamma)\bar{r}(s, a) + \gamma \max_{a'} M_{\text{target}}(s', a', g_+),$$

where g_+ is an artificial success goal and M is a successor-measure-like value. The core hypothesis was that this soft target would match normalized Q-learning in tabular sanity checks while avoiding sampled terminal-event variance. A secondary hypothesis, intentionally separated from the first, was that auxiliary real-state successor goals might improve low-capacity function approximation through shared representations.

5.2 How the autoresearcher intern approached the problem

The *autoresearcher intern* turned the idea into a ladder of experiments. The first experiment was the smallest possible diagnostic: a one-state MDP comparing sampled Bernoulli terminal targets with deterministic soft targets over several reward and discount values. This checked whether the two estimators have the same mean and different variance. A failure here would have immediately invalidated the estimator story.

The next stage tested tabular equivalence to normalized Q-learning. CliffWalking and small chain environments were used to verify scaling, terminal handling, and reward normalization. This was important because reward-to-goal conversions can appear successful while actually comparing values with inconsistent units. By using exact references, the system made the experiment a diagnostic rather than a benchmark race.

The middle stage tested propagation under coverage variation. RiverSwim-style chains examined whether soft updates help when reward propagation is long-horizon and whether any advantage depends on adequate coverage. The system separated adequate-coverage and starved-coverage regimes, which later prevented overclaiming. The result was not “soft always wins”; it was “soft helps under adequate coverage, but coverage-starved settings remain caveated.”

A later stage tested vector successor-measure correctness in a tiny FourRooms setting. Here the system checked that independent real-state goal slices and the artificial success-goal slice could be represented correctly. This was still a correctness check, not a shared-representation claim. Only after this did the system test a low-capacity shared low-rank model in which real-state auxiliary goals and the artificial reward head interacted through shared parameters.

Finally, when the low-rank auxiliary experiment produced negative transfer, the *autoresearcher intern* did not simply search for a better framing. It ran a predeclared repair diagnostic: terminal-only reproduction, combined auxiliary reproduction, loss-balanced combined training, and staged real-goal pretraining followed by artificial-goal fine-tuning. These repairs also failed to match terminal-only performance. The system then moved to synthesis rather than more sweeping.

6 Discussion

The case studies suggest that an autoresearcher intern can accelerate scientific work in ways that are not captured by benchmark accuracy alone. The most obvious benefit is implementation throughput: the system writes scripts, runs diagnostics, and maintains artifacts. But the deeper benefit is epistemic structure. It forces the question “what claim did this experiment test?” and requires the answer to be linked to metrics, baselines, and artifacts.

This matters because many scientific projects fail not through one dramatic error but through gradual overclaiming. A method works in one small setting, then the claim expands; a

Iterations	Question	Outcome	Claim boundary
0001	Do sampled and soft terminal targets match in mean but differ in variance?	Positive diagnostic	Supports variance-reduction mechanism only
0002–0004	Does scaled artificial-goal value match normalized Q in small tabular tasks?	Positive after repair checks	Requires audited normalization and terminal masks
0005–0007	Does soft learning help long-horizon propagation?	Coverage-qualified positive	Adequate coverage only; starved runs are caveated
0008	Are independent vector successor-measure goal slices correct?	Positive correctness check	Independent tabular goal slices only
0009–0010	Do real-state auxiliary goals help a shared low-rank reward head?	Negative transfer	Limited to tested rank-4 low-rank setup
0011–0013	Can evidence be packaged safely?	Internal draft	External use requires human review

Table 1: Evidence trajectory for Soft Reward-to-Goal Conversion. The *autoresearcher intern* did not return a single binary success/failure result; it narrowed the supported claim.

baseline is missing; a failure mode is noted but not integrated; an agent proposes another sweep because it can. The *autoresearcher intern* counters this with gates. The local reviewer can pass, weak-pass, fail, or require human input. The checkpoint policy can escalate to a stronger model. The metric ledger can preserve negative signals. The progress summary can state that the goal is not solved. These mechanisms make it easier for a human scientist to say: continue, stop, or reframe.

7 Limitations

This work has several limitations. First, we report live case studies rather than a controlled productivity evaluation. Second, both projects are computational reinforcement-learning prototypes. Third, the *autoresearcher intern* relies on LLM agents that can still fail. Fourth, GPT-5.5-Pro checkpoint supervision is not a substitute for expert scientific judgment. Finally, the current implementation is optimized for early-stage prototyping, not final scientific validation.

8 Conclusion

We introduced the *autoresearcher intern*, a repository-centered pipeline for evidence-gated human–AI scientific collaboration. The system operationalizes a bounded AI research intern: it translates rough ideas into small experiments, executes and reviews those experiments, maintains structured memory, escalates high-stakes decisions, and preserves both positive and negative evidence. Across two live reinforcement-learning research projects, the *autoresearcher intern* produced scoped positive claims, negative boundary results, and stopping decisions that prevented unsupported scaling. We believe this middle-ground autonomy, more capable than passive coding assistance, more accountable than unconstrained AI-scientist automation, is a promising design pattern for language models in scientific discovery.

References

- Reem Aleithan, Haoran Xue, Mohammad Mahdi Mohajer, Elijah Nnorom, Gias Uddin, and Song Wang. SWE-Bench+: Enhanced coding benchmark for llms, 2024. URL <https://arxiv.org/abs/2410.06992>.

- Alireza Ghafarollahi and Markus J. Buehler. SciAgents: Automating scientific discovery through multi-agent intelligent graph reasoning, 2024. URL <https://arxiv.org/abs/2409.05556>.
- Ali Essam Ghareeb, Benjamin Chang, Ludovico Mitchener, Angela Yiu, Caralyn J. Szostkiewicz, Jon M. Laurent, Muhammed T. Razzak, Andrew D. White, Michaela M. Hinks, and Samuel G. Rodriques. Robin: A multi-agent system for automating scientific discovery, 2025. URL <https://arxiv.org/abs/2505.13400>.
- Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. MAgentBench: Evaluating language agents on machine learning experimentation, 2023. URL <https://arxiv.org/abs/2310.03302>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world github issues?, 2023. URL <https://arxiv.org/abs/2310.06770>.
- Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI Scientist: Towards fully automated open-ended scientific discovery, 2024. URL <https://arxiv.org/abs/2408.06292>.
- Shuo Ren, Pu Jian, Zhenjiang Ren, Chunlin Leng, Can Xie, and Jiajun Zhang. Towards scientific intelligence: A survey of llm-based scientific agents, 2025. URL <https://arxiv.org/abs/2503.24047>.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023. URL <https://arxiv.org/abs/2302.04761>.
- Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Zicheng Liu, and Emad Barsoum. Agent Laboratory: Using llm agents as research assistants, 2025. URL <https://arxiv.org/abs/2501.04227>.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. URL <https://arxiv.org/abs/2303.11366>.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023. URL <https://arxiv.org/abs/2305.16291>.
- Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The AI Scientist-v2: Workshop-level automated scientific discovery via agentic tree search, 2025. URL <https://arxiv.org/abs/2504.08066>.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering, 2024. URL <https://arxiv.org/abs/2405.15793>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models, 2022. URL <https://arxiv.org/abs/2210.03629>.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models, 2023. URL <https://arxiv.org/abs/2305.10601>.
- Yanbo Zhang, Sumeer A. Khan, Adnan Mahmud, Huck Yang, Alexander Lavin, Michael Levin, Jeremy Frey, Jared Dunnmon, James Evans, Alan Bundy, Saso Dzeroski, Jesper Tegner, and Hector Zenil. Advancing the scientific method with large language models: From hypothesis to discovery, 2025. URL <https://arxiv.org/abs/2505.16477>.
- Tianshi Zheng, Zheyang Deng, Hong Ting Tsang, Weiqi Wang, Jiaxin Bai, Zihao Wang, and Yangqiu Song. From automation to autonomy: A survey on large language models in scientific discovery, 2025. URL <https://arxiv.org/abs/2505.13259>.

A Appendix: Stochastic Long-Horizon Goal Reaching Case Study

The second project asked whether stochastic variants of transitive value learning could improve long-horizon offline goal reaching. In ordinary deterministic goal-reaching tasks, a transitive or divide-and-conquer value update can use intermediate waypoints to propagate long-horizon reachability. The human scientist’s concern was that under stochastic dynamics, such updates may become overoptimistic: if an offline dataset contains lucky stochastic shortcut outcomes, a deterministic-style transitive backup may treat the shortcut as reliably good.

The charter therefore framed the project around three questions. First, can raw transitive value learning overestimate risky stochastic paths? Second, does a log-space version improve numerical stability and long-horizon propagation? Third, can stochastic calibration, successor-distance structure, posterior uncertainty, or history/context produce a distinct improvement over fair baselines?

The *autoresearcher intern* began with exact-DP tabular diagnostics. Early experiments confirmed that raw transitive value learning could prefer lucky stochastic shortcuts and that log-space transitive learning helped long-horizon propagation relative to Monte Carlo-only labels in censored settings. These were useful positive diagnostics: they identified a real failure mode and a useful baseline behavior.

The next experiments tested whether successor-distance or stochastic-calibrated variants produced distinct gains. They did not. Successor-distance variants that improved metrics were numerically equivalent to log-space transitive learning under the audit tolerance. A hand-shaped conservative penalty reduced one lucky-only overestimation case, but the rule was narrow. A more generic count-based uncertainty replacement reduced Q-overestimation modestly but did not fix the policy-level lucky-only failure.

The project then pivoted toward posterior and partial-observation settings. Posterior-transitive variants were compared to prior-matched posterior mean model-DP baselines and were found equivalent or near-equivalent. In aliased POMDPs, history/context helped strongly when cues were sufficient, but history-model DP explained the gains. The final GPT-5.5-Pro decision stopped the loop: the project had generated valuable diagnostics, but no reviewed experiment supported a distinct stochastic transitive-RL algorithmic advantage over the appropriate baseline.

Phase	What the intern tested	Decision-relevant outcome
Tabular stochastic shortcut	Raw and log-space transitive learning against exact DP	Raw optimism confirmed; log-space propagation useful
Successor-distance audit	Whether successor-distance variants add value beyond log-space transitive learning	Negative: improving variants were equivalent to log-space baseline
Uncertainty variants	Hand-shaped and generic conservative penalties	Narrow positive for hand-shaped rule; generic rule did not solve policy regret
Posterior/model baselines	Posterior-transitive variants vs. prior-matched model DP	Negative: model DP explained gains
Partial observability	History/context keys under aliased observations	History helps, but history-model DP explains all gains
Final check-point	Should the project continue?	Stop current line; reframe only with new human-approved question

Table 2: Stochastic Long-Horizon Goal Reaching case study. The pipeline produced useful diagnostics and a negative boundary result.

395 **B Appendix: Example Agent Prompts and Schemas**

396 The implementation uses short role prompts plus structured schemas. The exact wording
397 can change, but the important feature is that each role has a narrow contract.

398 **B.1 Supervisor prompt sketch**

399 You are SUPERVISOR in an automated research loop.
400 You are a planner and decision-maker, not an executor.
401 Read the charter, state, latest results, and latest reviews.
402 Choose continue, pivot, stop, needs_human, or pro_review.
403 If continuing, propose one small experiment that can finish
404 within the configured timeout. Reward evidence, not activity.
405 Return JSON matching decision.schema.json.

406 Required decision fields include decision, confidence, progress_score,
407 rationale, evidence, risks, and either a null or structured next_experiment.
408 The next_experiment must specify an experiment id, objective, hypothesis, success
409 criteria, failure criteria, tasks, required outputs, and estimated runtime.

410 **B.2 Executor prompt sketch**

411 You are EXECUTOR in an automated research loop.
412 Follow the supplied experiment plan.
413 Run only small-scale experiments suitable for quick validation.
414 Save exact commands, raw metrics, baseline metrics, artifacts,
415 a result JSON, and a summary Markdown file.
416 If the plan is impossible, write a failed result JSON explaining why.
417 Do not change the research goal or delete previous results.

418 The result schema requires fields for experiment_id, status, claim_tested,
419 commands_run, metrics, baseline_metrics, artifacts, interpretation,
420 known_failures, and next_questions. Artifact paths are validated before the loop can
421 continue.

422 **B.3 Reviewer prompt sketch**

423 You are REVIEWER in an automated research loop.
424 Audit the latest experiment. Check required files, artifacts,
425 baseline fairness, success criteria, failure criteria, cherry-picking,
426 overclaiming, and whether auto-continuation should be allowed.
427 Return JSON matching review.schema.json.

428 The review schema requires verdict, allows_auto_continue, reasons,
429 evidence_checked, required_fixes, and risk_flags. This makes reviewer
430 reflection inspectable rather than hidden in prose.

431 **B.4 Summarizer prompt sketch**

432 You are SUMMARY in an automated research loop.
433 Write a concise progress summary from reviewed evidence.
434 Separate confirmed findings from limitations and open questions.
435 Do not claim the research goal is solved unless charter success
436 criteria are satisfied by reviewed evidence.

437 Summaries include current status, experiment ledger, main findings, limitations and risks,
438 recommended next human decision, and files to inspect. They are used both for humans
439 and as compact input to a stronger model checkpoints.

440 **B.5 Stronger model checkpoint prompt sketch**

441 You are the external research supervisor.
442 Given the evidence packet, decide whether the project is making
443 real progress. Choose continue, pivot, stop, or needs_human.
444 Be skeptical. Reward evidence, not activity. If continuing, propose
445 one small experiment with success/failure criteria.
446 Return Markdown containing one fenced JSON block matching
447 pro_decision.schema.json, followed by a short explanation.

448 This role is intended as senior checkpoint supervision. It is invoked selectively rather than
449 at every local iteration.

450 **C Appendix: Example Charter**

451 Below is a shortened charter for the Soft Reward-to-Goal Conversion project. It illustrates
452 the level of human guidance expected before the agents begin.

453 Research goal:
454 Rapidly test whether a soft successor-measure reward-to-goal
455 conversion can preserve reward-to-goal equivalence while avoiding
456 sparse, high-variance sampled terminal success events.
457
458 Main hypothesis:
459 Training the artificial success-goal head with expected terminal mass,
460 $y(s,a,g+) = (1-\gamma) r_{\text{bar}}(s,a) + \gamma \max_{a'} M_{\text{target}}(s',a',g+)$,
461 will match normalized Q-learning in tabular sanity checks while
462 removing terminal-sampling variance.
463
464 Initial scope:
465 Start with tiny discrete environments: one-state diagnostics,
466 CliffWalking, RiverSwim, and tiny FourRooms. Do not run MuJoCo,
467 AntMaze, OGBench, image observations, or expensive neural training.
468
469 Primary metrics:
470 Target mean/variance; number of sampled g+ events per 10k transitions;
471 max absolute value error between scaled M_{plus} and normalized Q;
472 policy disagreement; Bellman error; average return.
473
474 Failure criteria:
475 Pause if terminal-only soft successor learning fails to match normalized
476 Q-learning, if reward normalization is ambiguous, if terminal masks are
477 wrong, if only training loss is reported, or if larger benchmarks are
478 attempted before tabular diagnostics pass.